Treehouse tETH EtherFi

March 27, 2026

Table of Contents

Summary 2

Overview 3

Issues 4

[WP-L1] `NavEtherFiWithdrawNft` Uses `unchecked` Subtraction for Fee That Can Exceed Withdrawal Amount 4

[WP-I2] `requestIds` are not deduplicated, which may lead to an inflated `nav` . 10

[WP-I3] Due to precision loss from division in `sharesForAmount(_amount)` within `eETH.transfer()` , the `etherFiEth.balanceOf()` corresponding to the shares actually received by the Strategy might be slightly less than the return value of `etherFiWeEth.unwrap()` as used directly in `EtherFiUnwrap._etherFiUnwrap()` . 12

[WP-I4] `NavErc20WithDebt` counts native ETH and `stETH` in the debt invariant 15

[WP-N5] The `PnlAccounting.deviation` comment is incorrect. It uses a 1e4 base, not 1e6. 18

Appendix 20

Disclaimer 21

Summary

This report has been prepared for Treehouse tETH EtherFi smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**Treehouse tETH EtherFi**
Codebase	**https://github.com/treehouse-gaia/tETH-protocol**
Commit	**6cc747d8664447bfe9922a59343ae7968987e6c9**
Language	**Solidity**

Audit Summary

Delivery Date	**March 27, 2026**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**5**

[WP-L1] `NavEtherFiWithdrawNft` Uses `unchecked` Subtraction for Fee That Can Exceed Withdrawal Amount

Low

Issue Description

`NavEtherFiWithdrawNft.nav()` replicates EtherFi's `getClaimableAmount()` logic to estimate NAV. It computes `uint fee = uint(request.feeGwei) * 1 gwei` and subtracts it from `amount` inside an `unchecked` block. If `fee > amount` (possible after a severe slashing event that shrinks `amountForShares` below the fee threshold, or for a micro-withdrawal against a non-zero fee), the subtraction silently underflows to near `type(uint256).max` .

Currently EtherFi does not charge fees (all on-chain requests show `feeGwei = 0`), but the `feeGwei` field exists in the struct and could be enabled in a future protocol upgrade.

Impact

The near- `uint256.max` intermediate value is then passed to `wstETH.getWstETHByStETH()` , which reverts with `ETH_TOO_LARGE` . This causes `nav()` **to permanently DoS** for any strategy holding an affected withdrawal NFT — making NAV calculation impossible until the NFT is removed from the inputs. Confirmed by fork test against mainnet wstETH (`0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0`).

PoC

Setup: mainnet fork. Only `WithdrawRequestNFT` is mocked (to inject a crafted `feeGwei`). Real `wstETH` (`0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0`) and real `LiquidityPool` (`0x308861A430be4cce5502d0A12724771Fc6DaF216`) are used via `vm.createSelectFork` .

```
// SPDX-License-Identifier: MIT
pragma solidity =0.8.24;

import "forge-std/Test.sol";

// ── interfaces ──────────────────────────────────────────────────────────
interface IWithdrawRequestNFT {
    struct WithdrawRequest { uint96 amountOfEEth; uint96 shareOfEEth; bool isValid;
uint32 feeGwei; }
```

```
    function getRequest(uint256) external view returns (WithdrawRequest memory);
    function ownerOf(uint256) external view returns (address);
}
interface ILiquidityPool { function amountForShare(uint256) external view returns
(uint256); }
interface IwstETH         { function getWstETHByStETH(uint256) external view returns
(uint256); }

// ── mock: ONLY mocked component ──────────────────────────────────────
contract MockWithdrawRequestNFT {
    struct WithdrawRequest { uint96 amountOfEEth; uint96 shareOfEEth; bool isValid;
uint32 feeGwei; }
    mapping(uint256 => WithdrawRequest) private _requests;
    mapping(uint256 => address)         private _owners;

    function setRequest(uint256 id, address owner,
                        uint96 amt, uint96 sh, bool valid, uint32 fee) external {
        _requests[id] = WithdrawRequest(amt, sh, valid, fee);
        _owners[id]   = owner;
    }
    function getRequest(uint256 id) external view returns (WithdrawRequest memory) {
return _requests[id]; }
    function ownerOf(uint256 id)    external view returns (address)                {
return _owners[id]; }
}

// ── buggy contract (verbatim original logic) ─────────────────────────
contract NavEtherFiWithdrawNftBuggy {
    IWithdrawRequestNFT public immutable withdrawRequestNFT;
    ILiquidityPool      public immutable liquidityPool;
    IwstETH             public immutable wstETH;
    error NotRequestOwner(); error InvalidRequest();

    constructor(IWithdrawRequestNFT n, ILiquidityPool l, IwstETH w) {
        withdrawRequestNFT = n; liquidityPool = l; wstETH = w;
    }
    function nav(address _target, uint[] calldata ids) external view returns (uint _nav)
{
        for (uint i; i < ids.length; ++i) {
            if (withdrawRequestNFT.ownerOf(ids[i]) != _target) revert NotRequestOwner();
            IWithdrawRequestNFT.WithdrawRequest memory r =
withdrawRequestNFT.getRequest(ids[i]);
            if (!r.isValid) revert InvalidRequest();
```

```
            uint amountForShares = liquidityPool.amountForShare(r.shareOfEEth);
            uint amount = r.amountOfEEth < amountForShares ? r.amountOfEEth :
amountForShares;
            uint fee    = uint(r.feeGwei) * 1 gwei;
            unchecked { _nav += amount - fee; }   // ← bug: silent underflow when fee >
amount
        }
        _nav = wstETH.getWstETHByStETH(_nav);
    }
}

// ── fixed contract ──────────────────────────────────────────────────
contract NavEtherFiWithdrawNftFixed {
    IWithdrawRequestNFT public immutable withdrawRequestNFT;
    ILiquidityPool      public immutable liquidityPool;
    IwstETH             public immutable wstETH;
    error NotRequestOwner(); error InvalidRequest();

    constructor(IWithdrawRequestNFT n, ILiquidityPool l, IwstETH w) {
        withdrawRequestNFT = n; liquidityPool = l; wstETH = w;
    }
    function nav(address _target, uint[] calldata ids) external view returns (uint _nav)
{
        for (uint i; i < ids.length; ++i) {
            if (withdrawRequestNFT.ownerOf(ids[i]) != _target) revert NotRequestOwner();
            IWithdrawRequestNFT.WithdrawRequest memory r =
withdrawRequestNFT.getRequest(ids[i]);
            if (!r.isValid) revert InvalidRequest();
            uint amountForShares = liquidityPool.amountForShare(r.shareOfEEth);
            uint amount = r.amountOfEEth < amountForShares ? r.amountOfEEth :
amountForShares;
            uint fee    = uint(r.feeGwei) * 1 gwei;
            uint claimable = fee >= amount ? 0 : amount - fee;   // fix
            _nav += claimable;
        }
        _nav = wstETH.getWstETHByStETH(_nav);
    }
}

// ── test ────────────────────────────────────────────────────────────
contract NavEtherFiPocTest is Test {
    address constant WSTETH         = 0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0; //
real, mainnet
```

```
    address constant LIQUIDITY_POOL = 0x308861A430be4cce5502d0A12724771Fc6DaF216; //
real, mainnet

    address attacker = makeAddr("attacker");
    MockWithdrawRequestNFT     mockNFT;
    NavEtherFiWithdrawNftBuggy buggy;
    NavEtherFiWithdrawNftFixed fixed_;

    function setUp() public {
        vm.createSelectFork(vm.envString("MAINNET_RPC")); // fork mainnet
        mockNFT = new MockWithdrawRequestNFT();
        buggy  = new NavEtherFiWithdrawNftBuggy(IWithdrawRequestNFT(address(mockNFT)),
ILiquidityPool(LIQUIDITY_POOL), IwstETH(WSTETH));
        fixed_ = new NavEtherFiWithdrawNftFixed(IWithdrawRequestNFT(address(mockNFT)),
ILiquidityPool(LIQUIDITY_POOL), IwstETH(WSTETH));
    }

    /// fee (1 gwei) > amount (0.5 gwei) → unchecked underflow → wstETH reverts
ETH_TOO_LARGE
    function test_FeeExceedsAmount_DoS() public {
        // Craft: amountOfEEth=0.5 gwei (5e8), feeGwei=1 → fee=1e9 > amount≈5e8
        mockNFT.setRequest(1, attacker, 0.5 gwei, 0.5 gwei, true, 1);
        uint[] memory ids = new uint[](1); ids[0] = 1;

        // Confirm the underflow wraps to ~type(uint256).max
        uint underflowed;
        unchecked { underflowed = uint(0.5 gwei) - uint(1 gwei); }
        assertGt(underflowed, type(uint128).max); // 115792...39936 ≈ type(uint256).max
- 4e8

        // buggy.nav() reverts: wstETH.getWstETHByStETH(~uint256.max) → ETH_TOO_LARGE
(DoS)
        vm.expectRevert();
        buggy.nav(attacker, ids);

        // fixed.nav() returns 0 cleanly (fee >= amount → claimable = 0)
        assertEq(fixed_.nav(attacker, ids), 0);
    }

    /// fee == amount → boundary: no underflow, both return 0
    function test_FeeEqualsAmount_BoundaryCheck() public {
        mockNFT.setRequest(2, attacker, 1 gwei, 1 gwei, true, 1); // fee == amount == 1
gwei
```

```
        uint[] memory ids = new uint[](1); ids[0] = 2;
        assertEq(buggy.nav(attacker, ids),  0);
        assertEq(fixed_.nav(attacker, ids), 0);
    }

    /// feeGwei == 0 (current EtherFi state) → both versions return identical NAV
    function test_NoFee_BothVersionsAgree() public {
        uint96 shares = uint96(ILiquidityPool(LIQUIDITY_POOL).amountForShare(1 ether));
        mockNFT.setRequest(3, attacker, 1 ether, shares, true, 0);
        uint[] memory ids = new uint[](1); ids[0] = 3;
        uint navBuggy = buggy.nav(attacker, ids);
        assertEq(navBuggy, fixed_.nav(attacker, ids)); // both agree when fee == 0
        assertGt(navBuggy, 0);                         // 813289544727858953 wstETH
    }
}
```

```
Ran 3 tests for NavEtherFiPocTest
[PASS] test_FeeEqualsAmount_BoundaryCheck()  (gas: 153079)
[PASS] test_FeeExceedsAmount_DoS()           (gas: 199554)
  underflowed value:
  115792089237316195423570985008687907853269984665640564039457584007912629639936
[PASS] test_NoFee_BothVersionsAgree()        (gas: 158325)
  wstETH NAV (both agree): 813289544727858953
```

Vulnerable Code

NavEtherFiWithdrawNft.sol

```
42          uint amountForShares = liquidityPool.amountForShare(request.shareOfEEth);
43          uint amount = request.amountOfEEth < amountForShares ? request.amountOfEEth
            : amountForShares;
44          uint fee = uint(request.feeGwei) * 1 gwei;
45
46          unchecked {
47            _nav += amount - fee;
48          }
49        }
```

Recommendation

Remove the `unchecked` block and add a floor-at-zero guard:

```
uint claimable = fee >= amount ? 0 : amount - fee;
_nav += claimable;
```

Status

✓ Fixed

[WP-I2] `requestIds` are not deduplicated, which may lead to an inflated `nav`.

Informational

Issue Description

1. `PnlAccounting.doAccounting()` indirectly calls the `nav` function, but only the Owner or Executor can invoke `doAccounting`.
2. There is a safeguard against `nav` deviation (it cannot exceed 2.5%).

Given that other invalid `requestId` inputs already include an owner validation check, we recommend adding a validation rule for duplicate `requestIds`. This will further prevent incorrect results caused by erroneous inputs.

NavEtherFiWithdrawNft.sol

```
26    /**
27     * @notice get sum of `_requestIds` EtherFi withdrawal NFT NAV of `_target`
28     * @param _target target address (must own the NFTs)
29     * @param _requestIds EtherFi withdrawal request Id array
30     * @return _nav nav in wstETH terms
31     */
32    function nav(address _target, uint[] calldata _requestIds) external view returns
(uint _nav) {
33      if (_requestIds.length == 0) return 0;
34
35      for (uint i; i < _requestIds.length; ++i) {
36        if (withdrawRequestNFT.ownerOf(_requestIds[i]) != _target) revert
NotRequestOwner();
37
38        IWithdrawRequestNFT.WithdrawRequest memory request =
withdrawRequestNFT.getRequest(_requestIds[i]);
39        if (!request.isValid) revert InvalidRequest();
40
41        // logic from WithdrawRequestNFT.getClaimableAmount(). Accounts for
potential slashing events.
42        uint amountForShares = liquidityPool.amountForShare(request.shareOfEEth);
43        uint amount = request.amountOfEEth < amountForShares ? request.amountOfEEth
: amountForShares;
```

```
44          uint fee = uint(request.feeGwei) * 1 gwei;
45
46          unchecked {
47            _nav += amount - fee;
48          }
49        }
50
51        _nav = wstETH.getWstETHByStETH(_nav);
52      }
```

Status

ⓘ Acknowledged

[WP-I3] Due to precision loss from division in `sharesForAmount(_amount)` within `eETH.transfer()`, the `etherFiEth.balanceOf()` corresponding to the shares actually received by the Strategy might be slightly less than the return value of `etherFiWeEth.unwrap()` as used directly in `EtherFiUnwrap._etherFiUnwrap()`.

Informational

Issue Description

In a single transaction where the `eETH` share price remains unchanged, calling `eETH.transfer()` using the amount returned directly by `etherFiWeEth.unwrap()` will result in the same downward rounding for the required shares; thus, the impact is minimal in this scenario.

However, if `etherFiWeEth.unwrap()` is combined with other eETH amounts and passed to `eETH.transfer()`, an exact division might unexpectedly trigger a revert.

https://github.com/treehouse-gaia/tETH-protocol/blob/a55d183bf8fa4ba2de98df5668582cd6e96638ea/contracts/strategy/actions/etherfi/EtherFiUnwrap.sol#L25-L46

```
25    function executeAction(
26      bytes calldata _callData,
27      uint8[] memory _paramMapping,
28      bytes32[] memory _returnValues
29    ) public payable virtual override returns (bytes32) {
30      Params memory inputData = parseInputs(_callData);
31      inputData.amount = _parseParamUint(inputData.amount, _paramMapping[0],
    _returnValues);
32
33      (uint eEthReceivedAmount, bytes memory logData) = _etherFiUnwrap(inputData);
34      emit ActionEvent(NAME, logData);
35      return bytes32(eEthReceivedAmount);
36    }
```

```
37
38    ////////////////////////// ACTION LOGIC //////////////////////////
39
40    function _etherFiUnwrap(Params memory _inputData) internal returns (uint
      eEthReceivedAmount, bytes memory logData) {
41        if (_inputData.amount == 0) revert ZeroAmount();
42
43        // No approval needed — unwrap burns caller's own weETH tokens
44        eEthReceivedAmount = IWeETH(etherFiWeEth).unwrap(_inputData.amount);
45        logData = abi.encode(_inputData, eEthReceivedAmount);
46    }
```

```
85        /// @notice Unwraps weETH
86        /// @param _weETHAmount the amount of weETH to unwrap
87        /// @return returns the amount of eEth the user receives
88        function unwrap(uint256 _weETHAmount) external returns (uint256) {
89            require(_weETHAmount > 0, "Cannot unwrap a zero amount");
90            uint256 eETHAmount = liquidityPool.amountForShare(_weETHAmount);
91            _burn(msg.sender, _weETHAmount);
92            eETH.transfer(msg.sender, eETHAmount);
93            return eETHAmount;
94        }
```

```
90        function transfer(address _recipient, uint256 _amount) external
          override(IeETH, IERC20Upgradeable) returns (bool) {
91            _transfer(msg.sender, _recipient, _amount);
92            return true;
93        }
94
          @@ 95,165 @@
166
167       // [INTERNAL FUNCTIONS]
168       function _transfer(address _sender, address _recipient, uint256 _amount)
          internal {
169           uint256 _sharesToTransfer = liquidityPool.sharesForAmount(_amount);
170           _transferShares(_sender, _recipient, _sharesToTransfer);
171           emit Transfer(_sender, _recipient, _amount);
172       }
173
```

```
@@ 174,180 @@

    function _transferShares(address _sender, address _recipient, uint256 _sharesAmount) internal {
@@ 183,190 @@
    }
```

```
    function sharesForAmount(uint256 _amount) public view returns (uint256) {
        uint256 totalPooledEther = getTotalPooledEther();
        if (totalPooledEther == 0) {
            return 0;
        }
        return (_amount * eETH.totalShares()) / totalPooledEther;
    }
```

Status

ⓘ Acknowledged

[WP-I4] `NavErc20WithDebt` counts native ETH and `stETH` in the debt invariant

Informational

Issue Description

`NavErc20WithDebt` appears intended to enforce a lending-position invariant:

> venue debt <= venue collateral

But the current implementation checks debt against `_nav`, and `_nav` includes assets that are not necessarily collateral for the lending venue.

```
    _nav += _target.balance; // native ETH counted immediately

    uint wip;
    uint wstETHBalance;
    for (uint i; i < _assetTokens.length; ++i) {
      wip = IERC20(_assetTokens[i]).balanceOf(_target);

      if (wip > 0) {
        unchecked {
          if (_assetTokens[i] == address(wstETH)) {
            wstETHBalance = wip;
          } else if (_assetTokens[i] == address(wstETH.stETH())) {
            _nav += wip; // stETH counted toward the invariant
          } else {
            _nav += (RATE_PROVIDER_REGISTRY.getRateInEth(_assetTokens[i]) * wip) / 1e18;
          }
        }
      }
    }

    for (uint i; i < _debtTokens.length; ++i) {
      ...
      if (_debtInEth > _nav) {
        revert InvariantViolation();
      }
    }
```

Why this is a problem

The debt check is effectively:

> debt <= all assets counted by this module

not:

> debt <= lending-venue collateral

So the invariant can be satisfied by:

1. native ETH sitting on the strategy, and
2. `stETH` included in `_assetTokens`, even if that `stETH` is idle in-wallet and not posted as collateral to the lending venue.

That makes the check semantically weaker than a strict collateral-vs-debt invariant.

Example

1. Strategy has 100 `stETH` idle in-wallet.
2. Strategy has 5 `variableDebtWETH`.
3. `_assetTokens` includes `stETH`.
4. The module counts that idle `stETH` toward `_nav`.
5. `InvariantViolation` does not revert, even though the check is being satisfied by non-collateral assets.

Recommendation

Tighten the invariant so that only lending-venue collateral is counted against lending-venue debt.

For example:

1. Remove `_target.balance` from this module.
2. Do not special-case `stETH` here unless `stETH` itself is explicitly intended collateral for the lending venue.
3. Keep raw / idle assets in a separate NAV module, rather than letting them participate in

this debt check.

Status

ⓘ Acknowledged

[WP-N5] The `PnlAccounting.deviation` comment is incorrect. It uses a 1e4 base, not 1e6.

Issue Description

250 actually represents 2.5%, not 0.025%.

The usage of `deviation` on L81 divides by `PRECISION` (defined as `PRECISION = 1e4` on L17).

https://github.com/treehouse-gaia/tETH-protocol/blob/a55d183bf8fa4ba2de98df5668582cd6e96638ea/contracts/periphery/PnlAccounting.sol#L13-L136

```
13  /**
14   * @notice Entry point for protocol PNL calculation
15   */
16  contract PnlAccounting is Ownable2Step, Pausable {
17    uint constant PRECISION = 1e4;
18

    @@ 19,30 @@

31
32    address public executor;
33    uint16 public deviation = 250; // 1e6 base. 250 == 0.025%
34    uint16 public cooldown = 3600; // in seconds
35    uint64 public nextWindow;
36    address public pauser;
37

    @@ 38,74 @@

75
76    /**
77     * @notice max PNL threshold per accounting window
78     * in terms of the underlying asset
79     */
80    function maxPnl() public view returns (uint) {
81      return (deviation * NAV_LENS.lastRecordedProtocolNav()) / PRECISION;
82    }
83
```

```
@@ 84,135 @@
136  }
```

Status

ⓘ Acknowledged

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.